Multi-Agent Systems 2025-26

Master on Artificial Intelligence (UDC, USC, UVigo)

First lab assignment: programming agents with JADE

Objective

The main objective of this lab is to get hands-on experience with agent development in JADE and experiment with different architectural design concepts.

Introduction

Inspired by the TileWorld[1] testbed, we will experiment on a simplified version we will refer to as "Item World".

In this world several agents will compete for resources which are randomly located on a *MxN* grid map. Competing agent's main task will be to explore this world and grab as many items as possible. For this purpose, at each *simulation round* a participant agent is allowed to request to move from its current position to the adjacent ones situated at *north, west, south* or *east* locations. Each time an agent moves towards a position holding an item, it scores one point. The world has a pre-configured fixed number of items, thus each time an agent reaches an item position, the item subsequently disappears, and a new one is randomly generated in another free position in the map. The simulation goes on until the pre-established number of specified simulation rounds has been reached.

The map, however, can also be populated with "trap positions". A competing agent would try to avoid navigating to these positions because one point will be subtracted from the agent's score counter, and the request to move will be eventually rejected (the agent will remain on its current position until the next turn). All requests for movement are controlled by a *Simulation Agent,* which in addition holds the state of the world and of each participant's perception of it.

Effectively, each competing agent's perception of the world is partially limited:

- For one, each participant is unaware of actions and the situation of other participant agents in the world.

- In addition, the world might periodically change (redistribution of positions of items and traps) after certain number of rounds. However, each participant agent will only be updated with the last actual state of the world according to *its commitment*. That is, an agent with commitment 10, for example, will only be updated with the actual situation of items and traps every 10 rounds. Hence, we could interpret *commitment* as a way to simulate the balance between reactivity and goal-directed behavior of an agent.

			1	
Ag 1	1	X	X	
			X	
1			***Ag 2***	

Figure 1. Representation of a possible situation in the item world. Two agents are situated on a 5x5 map where items are represented by "1s" and traps by "Xs". Each corresponding agent's next possible requests for movements are highlighted in gray. Notice an agent might also decide to "stay" on its current position. If *Ag2* would request moving "North", however, the action will not be allowed by the *Simulation Agent* and one point will be subtracted from its current "score".

We notice this situation leds to a *dynamical environment* in which conflicting situations might arise that will be handled by the Simulation Agent. This agent, therefore acts as referee processing each participant's requests for movement, checking its validity according to the actual simulation status, and returning the confirmation of movement, or not, to the corresponding participant, eventually updating its scoring points (if it corresponds). For example, one agent might request to move to a position which is already occupied by another agent. This request will be rejected by the Simulator Agent. Similarly, one agent with high commitment might request to move to a position which is supposed to contain an item, but this item might not be there anymore, either because preceding action of another agent, or because the world has been changed, the agent not yet being aware of it. And so on.

A more detailed functionality and communication protocol for each type of agent is described next:

<u>Simulator agent:</u>

-At start time, the Simulator Agent will initialize a random map according to pre-set configuration (i.e. number items, number traps, rescheduling periodicity) and register a "SimulatorService" with the Directory Facilitator (DF) agent provided by the JADE platform.

-It will wait for "REQUEST/join-simulation-request" messages (performative/conversationId) from possible participant agents, until the pre-set number of participants for the simulation has been reached. Each requester will also communicate its *commitment* parameter in the content of the message. Each request will be confirmed with "ACCEPT" or "REFUSE" (the latter if subsequent requests are submitted once the maximum number of participants has been already reached). If accepted, a "SimulationState" object will be sent attached to the response to the corresponding participant, to inform about the initial map configuration and its starting position.

-Once the required number of participants have been reached, the Simulator Agent will loop through all the registered participants, one-by-one, sending “REQUEST/request-action” messages, and waiting for their response for a maximum of 10 seconds. If no response by a participant is sent on that period of time, then it would be understood that its turn has expired for the current simulation round, proceeding with the next agent in the queue.

-Requests for action will be replied by participant agents with a “PROPOSE/request-action” message, attaching their requested next “Position”. The Simulator Agent will check the validity of this request and respond with “INFORM/update-state” message, attaching the effective next “SimulationState” for the requester accordingly.

-Once the pre-established number of simulation rounds has been reached, the Simulator Agent will send “INFORM/simulation-complete” messages to all the participants to inform about the end of the simulation.

-At the end of the simulation the participants’ final obtained scores will be shown through the command line.

<u>Participant agents:</u>

-At start time they will periodically check if the Simulator Agent is available in the Directory Facilitator.

-Once found, they will send a “REQUEST/join-simulation-request” message, waiting for corresponding response as described above. Their *commitment* configuration will be read as argument through the command line at their creation time.

-They will wait for “REQUEST/request-action” messages from the Simulator Agent, and according to their perceived “SimulationState”, decide what to do next (i.e. deliberate about the next requested “Position”), following the protocol described above. They will obtain the new “SimulationState” provided by the Simulator and update its internal perception accordingly.

-They will be listening all the time for possible “INFORM/simulation-complete” messages, informing about the end of the simulation, to stop their execution.

Task description:

Using JADE, you will be implementing different agents and run simulations in the described environment. You should develop <u>a total of three different types of participant agents</u>, each one following a different exploring strategy, and run simulations to analyze their effectiveness in obtaining the highest possible score.

Of them, one will implement a *random* decision strategy, basically consisting in requesting movements to random adjacent positions on each round. This agent will be used to set our “baseline score”.

The other two agent types of agents each will implement additional search strategies of your own choice. You might choose the strategy that you want, with the only restriction that

decisions can only be taken according to the corresponding world perception (i.e. *beliefs*). But notice this does not avoid the possibility to implement certain internal state and/or planning capabilities within the agent, if you opt to.

To facilitate the task, you will be provided with some template code with basic implementation the behavior for the Simulator Agent, and to support basic navigation and map generation utility. The use of this template code is optional. You might want to extend the basic functionality, or modify some parts, to fit your needs. You might even want to implement your own complete version from scratch, but be aware that this is not the main objective of the exercise. In particular, pay special attention not to alter the behavior encapsulated within the provided "*SimulationManagerBehaviour*" class, and be aware that evaluation of agent's performance by the teaching staff will be carried out using this code.

Deliverables and deadlines

The task will be carried out in groups of 2 people.

The deadline for submitting deliverables is February 26th at 23:59 p.m.

Deliverable materials will comprise all the implemented code and a separated report document. All materials will be included in one ZIP file, with its name clearly identifying all members of the group, to be sent either via email to: *s.suarez.marcote@udc.es* with the subject line "MAS2526_P1_*LastNameMember1_LastNameMember2*".

The implemented code should contain all source code (*.java), compiled files and possible additional resources (*.class, *.jar, etc.) and an additional "readme" file with clear instructions on how to execute it. In addition, an executable (script) file should be included that allows execution of one simulation example "in one go". Finally, it must be possible to instantiate all participant agents directly from the command line, e.g. like: "*java jade.Boot –agents agent1: AgentClass1*".

The report document will be delivered as PDF, with maximum length of 10 pages. It should comprise the following sections:

1. Title page, with summary and complete identification of all members of the group

2. Methods

> Describe the general behavior of the implemented participant agents, detailing their decision function, and justification of its adequacy (or not) to the described simulation environment.
>
> Think also about possible pre-conditions, state descriptions or specific agent parameterizations that might apply.
>
> Describe also the different (JADE) behaviors used to accomplish the general communication protocol described above, and possible specific ones for each particular type of agent (if that is the case).

Finally, for each one, discuss whether the overall implemented strategy resembles more a reactive vs. deliberative approach.

3. Simulation experiments

Taking as reference a map of size 10x10, with 5 items, run 10 repetitions of a 1000-round simulation (each one with a different random initialization), and construct the following table:

Type of agent	Average Final Score			
	Agent alone (commitment 1)	Agent alone (commitment 20)	All 3 agents together (commitment 1)	All 3 agents together (commitment 20)
Random agent				
Agent Type A				
Agent Type B				

for:

3.1. A “fixed” world without traps
3.2. A “fixed world with 10 traps
3.3. A “dynamic” world without traps
3.4. A “dynamic” world with 10 traps

and whereby as “fixed” we interpret a world without items/traps rescheduling (besides item regeneration after an agent reaching an item position), and by “dynamic” a world that carries out items/trap rescheduling after every 10 rounds.

4. Discussion of the results

4.1. Based on the results of execution examples, discuss the pros- and cons- of each of the implemented agents in the different situations.

4.2. How does the parameter commitment affect their behavior?

4.3. Which of the proposed types of agents would you consider it shows the best overall performance?

5. Conclusions

Here you should state your general conclusions. Comment as well on possible particular situations that might have grabbed your attention and/or could be object of further improvement or investigation

References

1. ME Pollack and M Ringuette (1988) "*Introducing the Tileworld: Experimentally evaluating agent architectures*", Proceedings of the AAAI Conference on Artificial Intelligence, pp. 183-189